

Project Setup Guide

Project Setup Guide

Monorepo · Docker · PostgreSQL · pgAdmin · Expo Go

PART 1 — Install Prerequisites

Step 1 — Install Homebrew (*Mac package manager*)

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Step 2 — Install Node.js 22

```
brew install node@22
node -v      # should print v22.x.x
```

Step 3 — Install pnpm

```
npm install -g pnpm
pnpm -v      # should print 9.x.x
```

Step 4 — Install Docker Desktop

Download from **docker.com** → Install → Open Docker Desktop → Wait for “**Engine running**”

Step 5 — Install Expo Go on your phone (*No Xcode / Android Studio needed*)

- iPhone → App Store → search **Expo Go** → install
- Android → Play Store → search **Expo Go** → install

Expo Go runs the app directly on your real phone over Wi-Fi. No simulator required.

PART 2 — Folder Structure

```
app project/
├── apps/
│   ├── mobile/      ← React Native (Expo) – the phone app
│   └── api/         ← Node.js + Express – the backend server
│       └── prisma/
│           ├── schema.prisma      ← database table definitions
│           └── migrations/       ← SQL files auto-generated by
Prisma
├── packages/
│   └── types/      ← shared TypeScript types (used by both apps)
├── docker/
│   ├── docker-compose.yml ← PostgreSQL + pgAdmin setup
│   └── .env         ← passwords & secrets
└── package.json
```

Folder	Purpose
apps/mobile	Expo React Native app (screens, components, stores)
apps/api	Express REST API (routes, services, Prisma)
packages/types	Shared TypeScript interfaces used by both apps
docker/	Docker Compose config + environment variables

PART 3 — Docker File (what's inside)

docker/docker-compose.yml runs two services — **PostgreSQL** (database) and **pgAdmin** (browser UI to view the database):

services:

```
# PostgreSQL – the actual database
postgres:
  image: postgres:16-alpine
  container_name: stickynotes-postgres
  environment:
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: postgres
    POSTGRES_DB: stickynotes
  ports:
    - "5432:5432"          # Mac port : container port
  volumes:
    - postgres_data:/var/lib/postgresql/data # data survives
      restarts

# pgAdmin – browser UI to manage the database
pgadmin:
  image: dpage/pgadmin4:8
```

```
container_name: stickynotes-pgadmin
environment:
  PGADMIN_DEFAULT_EMAIL: admin@admin.com
  PGADMIN_DEFAULT_PASSWORD: admin
ports:
  - "5050:80"          # open http://localhost:5050 in browser
depends_on:
  - postgres
```

```
volumes:
  postgres_data:
```

To open pgAdmin:

1. Go to `http://localhost:5050`
 2. Login: `admin@admin.com / admin`
 3. Add Server → Host: `postgres` · Port: `5432` · User: `postgres` · Password: `postgres`
-

PART 4 — Database Migrations (what they are)

Migrations are **auto-generated SQL files** created by Prisma when you change `schema.prisma`.

You never write them manually. The flow is:

Edit `schema.prisma` → run command → Prisma generates `migration.sql`
→ applies to DB

How to create a migration (when you add/change a table):

```
pnpm db:migrate
# Prisma asks for a name, e.g. "add_note_images"
# Creates:
  apps/api/prisma/migrations/20260318_add_note_images/migration.sql
```

What a migration file looks like (example from this project):

```
-- Creates the initial tables
CREATE TABLE "users" (
  "id"          TEXT      NOT NULL,
  "email"       TEXT      NOT NULL,
  "password"    TEXT      NOT NULL,
  "name"        TEXT,
  "createdAt"   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updatedAt"   TIMESTAMP NOT NULL,
  CONSTRAINT "users_pkey" PRIMARY KEY ("id")
);

CREATE TABLE "notes" (
  "id"          TEXT      NOT NULL,
  "title"       TEXT      NOT NULL,
```

```

    "content" TEXT,
    "color" TEXT NOT NULL DEFAULT '#FFF88',
    "userId" TEXT NOT NULL,
    "createdAt" TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    "updatedAt" TIMESTAMP NOT NULL,
    CONSTRAINT "notes_pkey" PRIMARY KEY ("id")
);

-- Prisma writes all of this automatically – you don't write SQL
  manually

```

Migrations in this project so far:

#	Migration Name	Table Affected	Columns Added / Changed
1	init	users (new)	id, email, password, name, createdAt, updatedAt
		notes (new)	id, title, content, color, userId, createdAt, updatedAt
		todos (new)	id, title, description, completed, userId, createdAt, updatedAt
2	add_password_reset_tokens	password_reset_tokens (new)	id, userId, token, expiresAt, createdAt
3	add_refresh_tokens	refresh_tokens (new)	id, userId, token, expiresAt, createdAt
4	add_events	events (new)	id, title, description, startDate, endDate, color, userId, createdAt, updatedAt
5	add_todo_status	todos (altered)	status — enum: TODO IN_PROGRESS COMPLETED

#	Migration Name	Table Affected	Columns Added / Changed
6	add_note_category	notes (altered)	category — TEXT, default 'General'
7	add_note_images	notes (altered)	images — JSONB, default '[]'

All database tables in this project:

Table	Purpose	Key Columns
users	Registered accounts	id, email, password, name
notes	User notes	id, title, content, color, category, images, userId
todos	Tasks / to-dos	id, title, description, completed, status, userId
events	Calendar events	id, title, description, startDate, endDate, color, userId
refresh_tokens	JWT login sessions	id, userId, token, expiresAt
password_reset_tokens	Forgot password OTP	id, userId, token, expiresAt

Prisma Schema (apps/api/prisma/schema.prisma) — this is what you edit, Prisma generates SQL from it:

```
model Note {
  id          String    @id @default(cuid())
  title       String
  content     String?
  color       String    @default("#FFFF88")
  category    String?
  images      Json      @default("[]")
  userId      String
  user        User      @relation(fields: [userId], references: [id])
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt
}
```

PART 5 — First-Time Project Start

```
# 1. Enter project folder
cd "/Users/vishakha/Documents/vishakha/app project"

# 2. Install all dependencies
pnpm install

# 3. Copy env file
cp docker/.env.example docker/.env

# 4. Generate a JWT secret – copy the output
openssl rand -base64 64

# 5. Open docker/.env and paste the output as JWT_SECRET value

# 6. Start PostgreSQL + pgAdmin
pnpm docker:up

# 7. Wait ~10 seconds, then run all migrations (creates all tables)
pnpm db:migrate

# 8. Find your Mac's local IP (needed for Expo Go on phone)
ipconfig getifaddr en0      # e.g. 192.168.1.10

# 9. Start the mobile app
pnpm dev:mobile
# Scan the QR code with your phone camera → opens in Expo Go
```

PART 6 — Daily Use

```
# Start everything
pnpm docker:up      # start PostgreSQL + pgAdmin
pnpm dev:mobile     # start Expo, scan QR with phone camera

# Stop everything
pnpm docker:down

# View API / database logs
pnpm docker:logs

# Reset database completely (wipes all data)
pnpm docker:reset
pnpm db:migrate     # re-run all migrations after reset
```

PART 7 — Useful URLs (when running)

URL	What it is
<code>http://localhost:3000</code>	Backend API
<code>http://localhost:5050</code>	pgAdmin (database browser UI)
<code>http://localhost:8081</code>	Expo web (browser preview)
Scan QR in terminal	Open app in Expo Go on phone